

Writeup

Senior Challenge 3 Vision AI target detection and follow

Everything below is written against what is actually in this repository's code, git history, and this session's real command output not from memory or assumption. Where something could not be independently verified, it is flagged as such rather than asserted.

1 Architecture: where this plugs into the broader system

The intended broader system, per `executor/schema.py`'s own docstring (written before this session, and confirmed accurate against how the executor is actually structured), is:

```
operator prompt -> LLM -> validated mission JSON -> deterministic executor -> simulator
/vehicle
```

This challenge's job was the last two arrows, for one new mission mode. The LLM only ever *proposes* a mission a list of steps like `{"action": "search_and_follow", "target_class": "person", "return_after_seconds": 30}` which is validated against a schema before it reaches `executor/executor.py`. `executor/schema.py`'s `require_fields()` is explicitly not that broader validator; it is a narrow, defensive per-action check, so a malformed step fails with a clear message instead of an `AttributeError` three calls deep in a control loop.

From there, everything is deterministic and auditable.

- `executor/executor.py`'s `MissionExecutor` is a plain dispatch loop: step dict in, look up the action string in `ACTION_REGISTRY`, call `.execute()`, log the result, stop on first failure. It has zero ROS, PX4, or LLM imports, by design, so it remains reusable if the vehicle stack underneath ever changes.
- `search_and_follow` (`executor/actions/search_and_follow.py`) is registered exactly like `takeoff/land` one more entry in `executor/registry.py`'s `ACTION_REGISTRY` dict, not a special-cased script. Adding a mission mode means writing one `ActionHandler` subclass and one registry line.
- Inside `search_and_follow`'s control loop, there is no LLM call anywhere `vision/follow_controller.py` is pure arithmetic (three independent proportional loops on bounding-box offset and size; see that file's docstring for the full derivation), driven only by `dt` and the current `Detection`. The LLM proposes the mission step once; the 10 Hz control decisions inside it are entirely deterministic Python.
- `vision/` (detector, controller, snapshot sender) has zero ROS or PX4 imports at all, so it is independently unit-testable (`tests/test_follow_controller.py`, `tests/test_detector.py`) and demoable against a real webcam (`tests/mock_demo.py`) with no simulator. `ros2_nodes/` is the only PX4/ROS-2-specific glue, isolated so the same `vision/` code would plug into a different vehicle stack for example a ground robot's Nav2 stack by writing a different vehicle-interface module.

2 What was attempted and solved

Challenge 3 is fully working, verified against live PX4 SITL and Gazebo, not just code review, twice.

1. **First integration pass** (this session): starting from a never-before-executed scaffold (see `README.md`'s "Verification history" it was originally written with no Python, ROS, or Gazebo available to test against), found and fixed three real integration bugs (below), then ran the actual mission end-to-end through the actual launch file (`ros2 launch sim/launch/search_and_follow.launch mission:=mission/mission_example.json ...`) and got:

```
step 0: success=True message=reached takeoff altitude 5.0m
step 1: success=True message=search_and_follow completed after 30.0s (final state:
        tracking)
step 2: success=True message=landed
```

2. **Second pass, after a refactor**: the `NAV_DLL_ACT` fix (bug 2 below) was originally applied by editing the stock PX4 airframe file directly; it was then refactored to use PX4's own `.post` autostart-override mechanism instead, so it survives a PX4-Autopilot upstream sync. To confirm the refactor did not change behaviour, PX4 and Gazebo were killed and restarted completely from scratch fresh process, fresh Gazebo world, target respawned, camera bridge relaunched and the identical mission was re-run through the identical launch command:

```
step 0: success=True message=reached takeoff altitude 5.0m
step 1: success=True message=search_and_follow completed after 30.0s (final state:
        tracking)
step 2: success=True message=landed
```

Identical outcome, with a full cold restart in between.

The evidence is not just log lines

Two additional, independent checks were run to confirm this is real closed-loop control and not merely printed numbers.

- **Position-telemetry correlation with setpoints.** During both runs, `/fmu/out/vehicle_local_position` (the EKF's actual estimated position physics state, not a command echo) was logged alongside `/fmu/in/trajectory_setpoint` (the commanded velocity) once per second for the full mission duration. The two are visibly correlated: once `search_and_follow` acquires the target, `sp_vel` becomes non-zero and time-varying (for example (N=-0.03, E=0.40, D=0.5) one second, different values the next), and the real position responds lateral (y) position drifted meaningfully (0.07 m to 0.75 m over four seconds in one run) and altitude (z) descended as the controller reacted to the target's low apparent position from altitude. Before acquisition (the `SEARCHING` state), `sp_vel` sat at exactly (0,0,0) and position held steady the contrast between the two phases in the same log is the proof.
- **A snapshot with the drone's own propeller blades visible in frame.** One saved snapshot from an actual mission run (`snapshots/20260714-175121_person.jpg`) shows dark propeller-blade shapes in the top corners of the image, alongside the annotated `person` 0.63 detection box. That is only possible if the frame came from the drone's own onboard camera mid-flight, not a static test image or a pre-recorded video.

3 Real integration bugs found and fixed

All four of these were discovered by actually running the system, not by code review the scaffold's own README.md, as originally written, explicitly said it had never been executed. Each was root-caused via direct log inspection, ldd, source reading, and live MAVLink/ROS 2 queries this session, not guessed at.

Bug 1 px4_msgs pinned to the wrong PX4 commit

Symptom: `ros2 topic echo /fmu/out/vehicle_status_v1` (and every other `/fmu/out/*` topic) printed nothing at all, indefinitely no error, no timeout message, nothing. `ros2 topic info -v` showed a matched publisher and subscriber with identical QoS on both sides, which normally means “should be working.”

Root cause: `~/px4_ros2_ws/src/px4_msgs` was checked out on main (`VehicleStatus.MESSAGE_VERSION = 4`, pinned via its own commit message to a PX4 commit that did not even exist in the local PX4-Autopilot checkout's history), while the actual running firmware was `v1.17.0-alpha1-117-ge0137fe7a7` (`VehicleStatus.MESSAGE_VERSION = 1`). The field layouts had diverged extra fields, a reordered enum between those two points in PX4's history. ROS 2/DDS discovery only checks topic name, type name, and QoS to declare a “match”; it does not check that the actual serialized byte layout agrees, so Fast DDS silently dropped every sample that failed its type-consistency check, with the drop surfacing nowhere a user would normally look.

Fix: `git checkout v1.17.0` in the `px4_msgs` repo (the tag matching this project's firmware release line), then rebuild. Confirmed immediately after: `ros2 topic echo` started printing real telemetry.

Bug 2 NAV_DLL_ACT blocking arming with no GCS attached

Symptom: with bug 1 fixed and telemetry flowing, `takeoff` still failed. Direct inspection of `/fmu/out/vehicle_status_v1` showed the OFFBOARD mode switch succeeding (`nav_state=14`) but arming being rejected (`arming_state` stayed DISARMED), and `pre_flight_checks_pass` stuck at `false` indefinitely even though every EKF, position, and battery health flag checked individually (via `/fmu/out/estimator_status_flags` and `/fmu/out/battery_status_v1`) was healthy.

Root cause: found by reading PX4's own source (`src/modules/commander/HealthAndArmingChecks/checks/rcAndDataLinkCheck.cpp`) after a MAVLink `VehicleCommand` probe returned `result=1` (TEMPORARILY_REJECTED) for the arm request. The check requires either RC input or an active GCS data-link connection whenever `NAV_DLL_ACT > 0` (the PX4 default is 2). This project has neither it flies entirely through the ROS 2 offboard/companion-computer link so the check could never pass at its default value.

Fix, and why it is not a simple one-line file edit: set `NAV_DLL_ACT = 0` via MAVLink `PARAM_SET` to confirm the hypothesis (arm immediately succeeded, `result=0` ACCEPTED). Made permanent first by editing the stock `4001_gz_x500` airframe file directly, then, once asked to make the fix durable against a PX4-Autopilot upstream sync, refactored into two separate `.post` override files (`4001_gz_x500.post`, `4010_gz_x500_mono_cam.post`) using PX4's own autostart-override mechanism in `rcS`, reverting the stock files to their original content. This refactor was verified to produce identical behaviour via a full cold restart (see “What was attempted and solved” above).

Bug 3 ros_gz_image linked against the wrong Gazebo generation

Symptom: the Gazebo camera sensor was confirmed producing real frames at roughly 30 Hz via `gz topic frequency`, but the bridged ROS 2 `/camera` topic never received a single message `ros2`

topic `hz /camera` hung indefinitely, and the bridge process's own log was full of `Unknown message type [8]/[9] lines` with no other error.

Root cause: `ldd $(ros2 pkg prefix ros_gz_image)/lib/ros_gz_image/image_bridge` on the apt-installed `ros-humble-ros-gz-image` package showed it linked against `libignition-transport11.so.11/libignition-msgs8.so.8` the older Ignition Fortress-generation Gazebo transport libraries. The actually-running simulator was Gazebo Sim 8.14.0 (Harmonic), speaking `gz-transport13/gz-msgs10`: a different, wire-incompatible major generation coexisting on the same machine as a separate package. The apt build was simply compiled against the wrong one.

Fix: a workspace-built `ros_gz_image` already existed at `~/px4_ros2_ws/install/ros_gz_image`, confirmed via the same `ldd` check to link `libgz-transport13.so.13/libgz-msgs10.so.10` (the correct, Harmonic-matching libraries). Sourcing that workspace's overlay (`source ~/px4_ros2_ws/install/setup` after `/opt/ros/humble`) before launching resolves `ros_gz_image` to the correct build. `/camera` then streamed real frames at roughly 2730 Hz.

Bug 4 numpy/cv_bridge ABI mismatch

Symptom: every run of `ros2_nodes/mission_executor_node.py` printed a NumPy ABI warning ("A module that was compiled using NumPy 1.x cannot be run in NumPy 2.2.6...") followed by an `AttributeError: _ARRAY_API not found` traceback from inside `cv_bridge`'s compiled extension, at import time. It looked non-fatal the mission still ran and returned real results which made it easy to dismiss, but that is exactly the risk: the warning being non-fatal for import does not guarantee `cv_bridge` produces correct image data on every conversion.

Root cause: the project venv had resolved `numpy==2.2.6` and `opencv-python==5.0.0.93`. `ros-humble-cv-bridge` (apt) is a compiled extension built against Ubuntu 22.04's stock NumPy 1.x C API, with which NumPy 2.0 broke ABI compatibility. `requirements.txt` already documented a `numpy<2` pin for exactly this reason, but `opencv-python>=5.0` hard-requires `numpy>=2` as a declared dependency, so pinning `numpy` alone was not sufficient: `pip install "numpy<2"` alone got immediately overwritten back to NumPy 2.x the moment anything re-resolved `opencv-python`.

Fix: pin both together `opencv-python>=4.9.0, <5` alongside `numpy<2` which resolves to `numpy==1.26.4 / opencv-python==4.9.0.80`, a mutually compatible pair. Verified not just by the warning disappearing, but by an actual `cv_bridge.cv2_to_imgmsg()` to `imgmsg_to_cv2()` round trip on a real array, checked byte-identical (`np.array_equal`) after the fix.

4 What was investigated and ruled out

Recorded for honesty, not presented as a fix. DDS domain-ID mismatch PX4's uXRCE-DDS client or the Micro-XRCE-DDS-Agent using a different discovery domain than the ROS 2 side's `ROS_DOMAIN_ID` was considered as a candidate explanation for the "zero messages" symptom that turned out to be bug 1. It was checked directly: `MicroXRCEAgent help` shows its discovery port defaults to 7400 (domain 0's SPDP port) and does not read `ROS_DOMAIN_ID` at all, so it is a real thing to be aware of if ever running with a non-zero `ROS_DOMAIN_ID` but in this environment every terminal had `ROS_DOMAIN_ID` unset (defaulting to 0) throughout, so there was no actual mismatch here. Recorded honestly as a hypothesis that was checked and excluded, not as a confirmed fix the real causes of that symptom were bug 1 above and, separately and earlier in the session, two PX4/Agent processes that had simply died and were not running at all.

5 How this would scale to harder, real-world problems

Real camera and gimbal hardware

`vision/detector.py` and `vision/follow_controller.py` have no simulator-specific assumptions baked in no hardcoded resolution, no Gazebo-specific coordinate handling so swapping `ros2_nodes/camera_subscription` topic source from a Gazebo bridge to a real camera driver's `sensor_msgs/Image` publisher should be closer to a configuration change than a rewrite. A real gimbal, though, would need a real design decision this project does not make: `follow_controller.py` currently commands vehicle yaw, vertical, and forward speed directly, assuming a fixed, body-forward camera (confirmed this session: the `x500_mono_cam` camera joint has zero mount tilt). A gimbal decouples camera pointing from vehicle heading, which changes the yaw-rate loop's job from "point the vehicle at the target" to "keep the gimbal pointed at the target, and only loosely slave vehicle yaw to it if at all" a real architectural change, not a tuning change.

PX4 hardware-in-the-loop and real flight

The `NAV_DLL_ACT=0` fix is specifically a SITL / companion-computer-only convenience flying a real vehicle with no GCS and no RC would be a genuine safety regression, not just a configuration nuisance. A real deployment should either restore a GCS or RC as an active safety link, or replace that check with a deliberate, reviewed failsafe policy (for example a real return-to-home-on-companion-computer-link-loss behaviour), not simply disable the check. Everything in `ros2_nodes/vehicle_offboard.py` the more-than-2 Hz heartbeat requirement, the arm/offboard-engage sequence is real PX4 API surface that hardware-in-the-loop and real flight also use, so that part transfers directly; it was confirmed working against real PX4 firmware behaviour this session, not just SITL-specific mocking.

Robustness to detection dropout and occlusion

Currently `lost_target_timeout_s` (default 2.0s) is the entire fallback: past that grace period the controller commands a hover and waits out `return_after_seconds`. A more robust version would need an active reacquisition search pattern for example a slow yaw sweep, or flying back toward the last known position rather than simply holding position, and probably a Kalman-filter-style position predictor to keep commanding reasonable motion through brief occlusions instead of freezing on the last filtered bounding box. See the deadband and EMA logic, and the `LOST` state, in `vision/follow_controller.py` for exactly what exists today.

Multi-target handling

`vision/detector.py`'s `pick_target()` is explicitly a single-target policy largest bounding box wins, described in its own docstring as a "reasonable default" with no notion of object identity across frames. Real multi-target tracking would need a tracker-with-identity, for example a simple centroid tracker or a proper ByteTrack/SORT-style association step, layered between `Detector.infer()` and `FollowController.update()`, so "follow this specific person" stays well defined even when a second person of the same class enters frame.

Network and comms degradation in the field

The heartbeat-on-its-own-timer design (see `README.md`, "Design decisions") already protects against one class of degradation: a slow vision-inference frame can never itself stall the safety-critical `OffboardControlMode` stream. It does not protect against the companion-computer link itself degrading or dropping for example a real radio link with packet loss and latency, as opposed to a same-

host DDS bridge. That would show up as PX4's own `offboard_control_signal_lost/gcs_connection_lost` failsafe flags (both directly observed in this session's `/fmw/out/failsafe_flags` during idle periods between missions) tripping mid-flight rather than between missions, and would need a deliberate decision about what PX4 failsafe action to configure for that case in a real deployment. This project never had to make that decision, because SITL's loopback DDS link never actually degrades.